

Minor Project Mid-Term Report

Information Extraction and Knowledge Graph Construction in Legal Domain using LLMs

Submitted in partial fulfillment of the requirement for the degree of
Master of Science in Computer Science

Submitted by

Harsh Yadav

23234747026

Vishesh Goyal

23234747061

Under the Guidance of

Prof. Vasudha Bhatnagar and Dr. Vikas Kumar

Department of Computer Science, University of Delhi



Department of Computer Science

Faculty of Mathematical Sciences

University of Delhi

Academic Year 2024-2025

Declaration

This is to certify that this project report titled “**Information Extraction and Knowledge Graph Construction in Legal domain using LLMs**” is being submitted to the Department of Computer Science, University of Delhi, New Delhi in partial fulfillment of the requirement for the award of the degree of M.Sc. (Computer Science). It is a record of bonafide work carried out by us under the supervision of Prof. Vasudha Bhatnagar and Dr. Vikas Kumar. The matter embodied in the declaration has not been submitted in part or full to any university or institution for the award of any degree or diploma.

Date: September 30, 2024

Harsh Yadav (23234747026)

Place: New Delhi

Vishesh Goyal (23234747061)

Certificate

This is to certify that this project report titled “**Information Extraction and Knowledge Graph Construction in Legal domain using LLMs**” submitted to the Department of Computer Science, University of Delhi, for the award of the degree of M.Sc. (Computer Science), is a research work carried out by Harsh Yadav (23234747026) and Vishesh Goyal (23234747061) under the joint supervision of Prof. Vasudha Bhatnagar and Dr. Vikas Kumar. This work has not been submitted in part or full to any other University or Institution for the award of any degree or diploma, to the best of our knowledge.

Prof. Vivek Kumar
(Head)

Prof. Vasudha Bhatnagar
(Supervisor)

Dr. Vikas Kumar
(Supervisor)

Acknowledgement

We would like to extend our profound gratitude to our supervisor Prof. Vasudha Bhatnagar and Dr. Vikas Kumar for their interest, guidance and valuable suggestions throughout this project. We feel honoured and privileged to work under their constant supervision. Without their advice, constructive ideas, positive attitude, and continuous encouragement, it would not have been possible to make such progress in the designated time frame.

Harsh Yadav
(23234747026)

Vishesh Goyal
(23234747061)

Contents

1	Introduction	5
1.1	Understanding Knowledge Graphs	5
1.2	Understanding Information Extraction	6
1.3	Domain Specific Knowledge Graphs: The Legal Domain	7
1.3.1	Applications in the Legal Domain	7
1.4	Challenges in Legal Knowledge Graph Construction	8
1.5	Organization of the Report	8
2	Literature Review	10
2.1	Traditional Approaches	10
2.2	Open Information Extraction(OIE)	11
2.3	Leveraging Large Language Models for OIE	13
3	Experimental Analysis	14
3.1	Experimental Setup	14
3.1.1	Purpose	14
3.1.2	Dataset(s)	14
3.1.3	Hardware and Environment	15
3.2	Methodology	15
3.3	Experiments	16
3.3.1	Triplets Generation of REBEL Dataset, Testing the Pipeline . . .	17
3.3.2	Triplets Generation of the Judgement, Comparison of the LLMs .	17
3.3.3	Triplets Generation of 10 Judgment Summaries	21
4	Conclusion and Future Work	22
	References	23
A	Experiments Output Files	26

Chapter 1

Introduction

Information extraction and knowledge graph construction are fundamental processes in the field of natural language processing (NLP). Information extraction focuses on deriving structured relationships such as triplets consisting of entities and their connections—directly from unstructured text. For instance, the phrase "Hayao Miyazaki was born in Tokyo" can be translated into the triplet (Hayao Miyazaki, born in, Tokyo).

This structured format is essential for knowledge graph construction, which organizes such information into a cohesive framework that represents knowledge through nodes (representing concepts) and edges (representing relationships), facilitating enhanced data retrieval and semantic understanding. The significance of these processes is underscored by its applications across copious domains, including artificial intelligence, finance, healthcare and legal.

Despite these advancements, limitations remain, particularly in accommodating dynamic environments where new entities and relationships emerge, necessitating further research into open-world techniques for information extraction and knowledge graph construction. As interest in generative AI and large language models grows, the landscape of information extraction and knowledge graph construction continues to evolve.

The integration of these technologies is expected to enhance the quality and utility of knowledge graphs, thereby facilitating their application in increasingly complex and data-rich environments. These developments highlight the potential for transformative advancements in how information is represented and utilized across diverse sectors.

1.1 Understanding Knowledge Graphs

As noted previously in introduction that knowledge graphs are labelled graphs consisting of nodes(representing concepts) and edges(representing relationships). Previous work on

knowledge graph creation can be divided majorly into two categories: (i) Domain specific knowledge graphs and (ii) literature knowledge graphs:

Domain specific knowledge graph: As the name suggests these graphs are domain specific, it means the text used in construction of these graphs belongs to a specialised domain like biology, physics, law, etc. One of the seminal works on domain specific knowledge graph construction done in the field of biomedical was done using rdf extraction from excel sheets in [1].

Literature Knowledge graph: They act as storage mechanisms for representing concepts and relations in the literature associated with some domain of interest. A well known literature knowledge graph is presented in [2]. Another well-known literature knowledge graph was created by Microsoft known as *Microsoft academic knowledge graph* in [3]

In this report we will be focusing on domain specific knowledge graph construction specializing in legal domain.

1.2 Understanding Information Extraction

As discussed KGs are powerful tools for visualizing information in an interconnected way. But at the heart of every KG lies, how we extract information from an unstructured text corpus. Information extraction provides us with models to extract structured data from an unstructured text corpus.

There are two most discussed approaches for Information extraction. Traditional information extraction approaches which require a pre-defined schema, an agreed set of specific concept types, and relation types as input. They generally require domain based supervised learning approaches. However, training data and domain specific schema are not available in majority cases like in ours where obtaining schema and dataset for legal domain is a particularly arduous task. Therefore, the alternative approach is utilizing open information extraction models which are domain independent and pretrained on general datasets thus they don't require any predefined schema.

Triplet Extraction using Information Extraction:

Triplet extraction involves three main steps (i) Named entity recognition (ii) Relationship extraction (iii) Event extraction. **Named Entity Recognition(NER)** includes task entity identification and entity typing. The former task involves identifying span of entities and the latter task focuses on containerizing these identities. The second step, **Relationship Extraction(RE)** involves identifying relationship between different entities

which often act as edges of knowledge graphs. The last step, **Event Extraction (EE)** aims to identify and classify the triggered word and type that most clearly represents the occurrence of an event.

1.3 Domain Specific Knowledge Graphs: The Legal Domain

Talking about the need for knowledge graph construction in legal domain, legal professionals, whether they are lawyers, judges, or legal researchers, they all rely on vast volumes of information to make informed decisions, build persuasive arguments, and navigate the intricate web of laws, regulations, and case precedents. In the era of technology-driven innovation, legal knowledge graphs are emerging as a powerful tool that promises to revolutionize the practice of law.

Building a knowledge graph for the legal domain requires extracting useful information from legal documents. The structure of extracted information is mostly in the form of triplets as they are easily convertible to knowledge graphs. Extracting triplets involves same steps as described in section above.

1.3.1 Applications in the Legal Domain

In the legal domain, knowledge graphs can be used to significantly improve efficiency, transparency, and decision-making process. Some of the potential applications include:

Relation Extraction and Inheritance Dispute: Inheritance disputes are the types of cases in Civil Law that focuses on the distribution of inheritance rights. Therefore, identifying the relationship between the parties is vital, as those who have the closest relationship with the deceased can get more assets. Towards this goal, relation extraction in inheritance dispute cases can provide the reason for judgment results and can improve the performance.

Contract Analysis: Contracts are unstructured documents that contain in them numerous relationships between parties, obligations, and rights. We can extract triplets from these documents for example:

1. **Subject:** Company A
2. **Predicate:** Is liable for
3. **Object:** Payment of damages to Company B

and then construct a knowledge graph out of these triplets. By doing so, automated sys-

tems can scan and analyse these legal documents for inconsistencies, potential breaches, or clauses that require attention.

Event Timeline Extraction and Judgment Prediction of Criminal Cases:

In many criminal cases, multiple parties are often involved in a group crime. To decide who should be primarily responsible for the crime, we need to determine what everyone has done throughout the case, and the order of these events is also essential. For example, in the case of crowd fighting, the person who fights first should bear the primary responsibility. As a result, a qualified event timeline extraction model is required for judgment prediction of the criminal cases.

1.4 Challenges in Legal Knowledge Graph Construction

While knowledge graphs hold significant potential for the legal field, there are challenges to overcome:

Data Privacy: Legal documents often contain sensitive information, so building and querying knowledge graphs must be done with strict privacy safeguards.

Complexity of Legal Language: Legal texts are often complex and ambiguous, making it difficult to accurately extract entities and relationships.

Maintenance and Updating: The law is constantly evolving, so keeping a legal knowledge graph up to date with new rulings and regulations is an ongoing task.

Despite these challenges, the potential of knowledge graphs to revolutionize the legal domain is undeniable. As NLP techniques and AI-driven reasoning improve, the accuracy and usability of legal knowledge graphs will increase, leading to more efficient legal research, contract management, and compliance.

1.5 Organization of the Report

The rest of the report is organized as follows: In **Chapter 2** we have provided an in-depth analysis of relevant research papers that have informed and influenced the current project. In **Chapter 3**, we have presented our experimental setup and the analysis of the experiments where we report our findings when using different large language models. We have also discussed the generation of triplets from 10 legal judgment summaries in this chapter. In **Chapter 4** we put forward our conclusions and possibilities of future work related to the project.

Appendix A contains the datasets used in the experiment and the triplets generated corresponding to them.

Chapter 2

Literature Review

In this chapter we have presented a literature review of Information Extraction(IE) systems and knowledge graph construction. Starting from simple schema based IE systems all the way to LLM based generative IE systems which are used to construct KGs.

2.1 Traditional Approaches

Traditional approaches to IE helps us in answering only very narrow well defined requests over a predefined set of target relations over a very small set of domain specific text corpus.

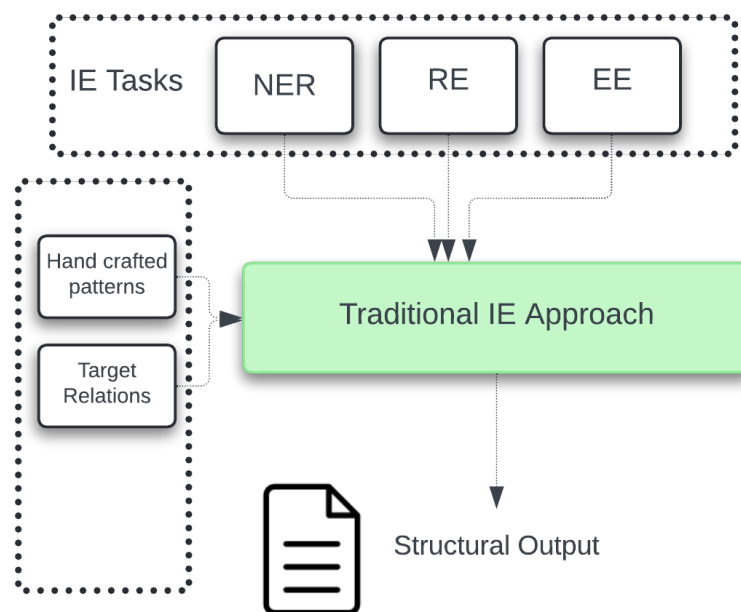


Figure 2.1: Traditional IE system.

Their input is predefined schema and hand crafted patterns learned from domain specific supervised learning. Table below shows a summary of some important IE algorithms based on traditional approaches

Feature	Learning Dictionaries for Information Extraction by Multi-Level Bootstrapping [4]	Snowball: Extracting Relations from Large Plain-Text Collections [5]	The PageRank Citation Ranking: Bringing Order to the Web[6]
Purpose	Learning dictionaries (lexicons) for information extraction from text	Extracting relations between entities from large plain-text datasets	Ranking web pages based on their importance through link analysis
Input Data	Small set of seed words or patterns, and unannotated text	Small set of seed relational tuples and large unannotated text corpora	Web pages and hyperlinks between them
Methodology	Bootstrapping to iteratively expand dictionaries based on pattern extraction	Bootstrapping and pattern-based relation extraction	Link-based ranking using the web's hyperlink structure
Key Technique	Multi-level bootstrapping: extracting patterns and refining dictionaries iteratively	Pattern recognition and extraction of relational tuples	Graph-based algorithm: link analysis to propagate "importance"
Application Domain	Information extraction, Named Entity Recognition (NER), domain-specific lexicons	Relation extraction (e.g., person-organization, organization-location)	Web search engines, ranking of search results
Output	A dictionary of entities, patterns, or lexical entries for information extraction	Set of relational tuples between named entities	Ranked list of web pages
Advantages	Iterative refinement with minimal supervision	Scalable to large corpora	Efficient and scalable for the web, with global importance
Limitations	Extensive human involvement; may work only for small, homogeneous corpora	May extract incorrect relations if seed or pattern is too generic	Can be manipulated (link farming); doesn't consider content relevance

Table 2.1: Comparison of Information Extraction and Ranking Algorithms

The main drawback of these systems was that they required a lot of human intervention and predefined domain specific schemas and datasets.

2.2 Open Information Extraction(OIE)

In 2007, the term "Open Information Extraction" was introduced by Banko et al. (2007)[7] at the University of Washington. This laid the groundwork for extracting relational tuples from text without predefined relations.

Their **TEXTRUNNER** system represents a significant advancement in OIE, in which the system makes a single, data-driven pass over the entire corpus and extracts a large set of relational tuples, without requiring any human input. They also identified a major challenges faced by OIE systems:

Automation: Open IE systems must automate relation extraction, which in traditional IE systems are hand labeled inputs.

Corpus Heterogeneity: Open IE systems must be able to work on heterogeneous corpora in contrast with traditional IE systems which are only able to work on small homogeneous corpora.

Efficiency: Open IE systems must be computationally efficient in order to scale to large amounts of text.

All these criteria were first implemented in the open IE system **TEXTRUNNER**. This seminal work triggered a lot of research effort in this area although not all of them end following the initial guidelines.

History of Open Information Extraction and Knowledge Graph Construction systems:

TEXTRUNNER [7] was the seminal work in the field of OIE systems. It introduces the concept of first open IE system which is scalable too. It worked by extracting relational tuples from the text corpus without requiring any human input.

After this in 2010 **WOE** [8] was introduced which was a huge improvement over **TEXTRUNNER** in precision and F1 score. It used a novel self-supervised algorithm which led to increase in its performance. However, the weakness of **WOE** was that it only extracted relations mediated by verbs. This weakness was overcome by **Open Language Learning for Information Extraction(OLLIE)** [9] which also included relations mediated by nouns and adjective in order to improve performance. The OIE systems discussed above faced the issue of generating overly specific representations, this problem however was solved by **Minimizing Facts in Open Information Extraction(MinIE)** [10] . Introduced in 2017 it focused in extracting much more richer relations from text.

There are broadly two approaches used by approaches i.e. (i) discriminative and (ii) generative. In this report we won't be delving deep into these approaches rather we will provide a general overview. The approaches defined in [11] are all discriminative, where as [12] talks about generative approaches for OIE system.

For a discriminative model objective is to maximize the likelihood of the data. It comprises of taking an annotated sentence x and a collection of overlapping triplets,

$t_j = (s, r, o) :$

$$P_{cls}(t|x) = \prod_{(s,r,o) \in t_j} p((s, r, o)|x_j)$$

Another method of discrimination involves generating tags using sequential tagging for each position i . For a sentence x consisting of n words, n different tag sequences are annotated based on the “BIESO” (Begin, Inside, End, Single, Outside) notation schema. During the training of the model, the objective is to maximize the log-likelihood of the target tag sequence by utilizing the hidden vector h_i at each position i :

$$P_{tag}(y|x) = \frac{\exp(h_i, y_i)}{\exp(\sum_{y'} \exp(h_i, y'))}$$

2.3 Leveraging Large Language Models for OIE

In recent years instead of using discriminative approach the use of LLMs for OIE and KGC has garnered a lot of attention. The work of pretrained LLMs is to extract information particularly in form of triplets which involves identifying named entities, extracting relations and extracting events as described in chapter 1. Recent attempts to automate KGC have employed large language models (LLMs) in view of their remarkable natural language understanding capabilities.

LLM-based KGC methods employ various innovative prompt-based techniques, such as multi-turn conversation [13] and code generation [14], to generate entity-relation triplets that represent the knowledge graph. However, these methods are currently limited to small and domain-specific scenarios — to ensure the validity of generated triplets, schema information (e.g., possible entity and relation types) has to be included in the prompt. Complex datasets (e.g., Wikipedia) typically require large schemas that exceed the context window length or can be ignored by the LLMs [15].

Chapter 3

Experimental Analysis

3.1 Experimental Setup

3.1.1 Purpose

The purpose of this report is to compare and contrast the ability of copious types of pretrained LLMs along with EDC[16] on a domain specific (legal) dataset.

3.1.2 Dataset(s)

- **REBEL Dataset:** The original test partition of REBEL[17] comprises 105,516 entries. We selected a random sample of 1000 text-triplet pairs. This subset induces a schema with 200 distinct relation types.

- **The Judgment:** For comparing multiple LLMs, we chose a preprocessed Judgment titled “**Indian Oil Corporation v NEPC India Limited and Others - Supreme Court of India**”. The whole judgement comprised of 64 examples, spread across 366 lines/sentences. One example sentence of the judgment is:

“The appellant (Indian Oil Corporation, for short ‘IOC’) entered into two contracts, one with the first respondent (NEPC India Ltd) and the other with its sister company Skyline NEPC Limited (‘Skyline’ for short) agreeing to supply to them aviation turbine fuel and aviation lubricants (together referred to as ‘aircraft fuel’).”

- **10 Judgment Summaries:** The Information about summary filename, number of sentences in each summary is mentioned in Figure-3.4.

3.1.3 Hardware and Environment

During our experiments, we ran the EDC pipeline on the two servers simultaneously. We downloaded the repository and then setup the environment to both our local servers. The specifications of both the servers are as follows:

SERVER-I (System1)

Component	Details
RAM	512 GB
Processor	Intel® Xeon® Gold 6326 CPU @ 2.90 GHz (2 processors)
OS Type	64-bit operating system, x64-based processor
OS Name	Windows Server 2022 Standard
WSL Version	Ubuntu 20.04.6 LTS

Table 3.1: System Configuration Details for Server-I

SERVER-II (System2)

Component	Details
RAM	256 GB
Processor	Intel® Xeon® Gold 6226R CPU @ 2.90 GHz $\times$ 64
Graphics	NVIDIA Corporation GA102GL [RTX A6000]
OS Type	64-bit
OS Name	Ubuntu 22.04.4 LTS

Table 3.2: System Configuration Details for Server-II

Reasons for using the different servers at the same time were to compare the LLMs inference time for triplet generation on different machines, and to parallelize the process for saving time. There are few examples in our corpus that we ran on both the servers, and few that we ran on either one of them only.

3.2 Methodology

Our pipeline is based on the paper titled “Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction” [16]. In this section we explain the pipeline in detail.

Generating triplets traditionally was a complex and time-consuming task to be done by a human manually. To automate this process, [16] introduced a three-phase framework named Extract-Define-Canonicalize (EDC), which utilizes the power of pre-trained LLM’s like ChatGPT and Mistral to generate the triplets.

Additionally, [16] solves the key issue in traditional triplet generation task, i.e the compulsory need of providing a valid schema in the Large Language Model prompt to generate corresponding triplets. This can become worse if the schema is complex and exceeds the LLM’s context window length. Therefore, [16] handles this by incorporating a technique which can work in both the scenarios; if we have a valid schema, we can input it with the data and if we do not have an existing schema, it can generate the schema of its own using Large Language Model itself. In the final phase, a RAG type technique is used for further refinement of the triplets iteratively using Schema Retriever.

What is EDC?

EDC stands for Extract-Define-Canonicalize. These steps can be understood as:

- **Open Information Extraction (OIE)**: It is responsible for extracting the triplets from the dataset. One triplet is a pair of Subject, Relation, Object.
- **Schema Definition (SD)**: This phase is responsible for defining every component of the schema.
- **Schema Canonicalize (SC)**: This phase is responsible for eliminating the redundancy and ambiguity in the extracted triplets by utilizing the schema definitions generated above. These definitions are used to find the closest entity/relation type candidates (via a vector similarity search) that the LLM can then reference to canonicalize a component.

According to [16], the following models should be utilized corresponding to these steps:

- **OIE**: GPT-4 / GPT-3.5 Turbo / Mistral-7B-Instruct-v0.2
- **SD**: GPT-3.5 Turbo
- **SC**: GPT-3.5 Turbo & Mistral-7B-Instruct-v0.2 (for vector similarity search)

However, due to the unavailability of `OPENAI KEYS`, we used only open source models (`Mistral-7B-Instruct` / `LLama 2.1-7B` / `LLama 3.1-8B`). In all the three steps same language model has been used.

3.3 Experiments

We first ran the `example.txt` file to test the pipeline on our local system¹. Once Confirmed, we moved with REBEL dataset provided with repository itself.

3.3.1 Triplets Generation of REBEL Dataset, Testing the Pipeline

LLM - *Mistral-7B-Instruct-v0.2*

Initially, we extracted triplets of 19 sentences from the REBEL dataset. Followed by 100 sentences, and finally on 1000 sentences. Moving further, we ran evaluation script to match the result of 1000 sentences with the scores claimed in the paper. But, Terminal prompted the following error on both the machines: “ OOM(out of memory):Killed process issue”, where the OS automatically kills the process, if the system is under heavy memory pressure.

We suspected that the script is inefficient and has memory leaks, which is continuously increasing memory usage, and after reaching to the authors through the mail, they admitted this issue with the script.

Meantime, we discovered that tweaking the `Max_length_Diff` Parameter, can deliver the results on 1000 sentences in no-time, when setting the parameter=1, and can be set up to 4 without any fail (with increase in time for each succession). `Max_length_Diff` Parameter was skipping results that had a huge length difference with the reference. Setting this to 4 for REBEL dataset, it cut down evaluation time to ~5 mins and ~50 entries are skipped out of 1000. But, [16] did not skip any results during evaluation, and that’s the reason, it took them 3 days to evaluate 1000 sentences with 1024 GB RAM server. Our results were coming to be ~0.17, which was too low in compared to what [16] was claiming. We figured out two root causes for this mismatch:

1. Using Mistral-7B-Instruct-v0.2 in all the three steps, instead of GPT3.5 Turbo in SD and SC.
2. We were using `Max_length_Diff` parameter, which authors was not.

3.3.2 Triplets Generation of the Judgement, Comparison of the LLMs

Considering these things, we moved forward to our goal of utilizing this pipeline for Legal purposes. Our project outline was to select a Legal document (Judgment in our case), compare it with several LLMs on the following bases:

- Inference Time (monitored)
- Quality of Triplets generated (human supervised)

and finally choose the model that has the lowest triplet generation time and generates the best quality triplets among all. Evaluation with ground triplets was not done, due to

unavailability of such triplets corresponding to our Indian Legal Judgment’s dataset.

Comparison

Models	Time (mins)			Total time (mins)	Total time (hours)
	OIE	SD	SC		
Mistral-7B-Instruct-v0.2	315	495	794	1604	26.7
LLama2-7b-chat-hf	211	437	703	1351	22.5
LLama3.1-8b-Instruct (Target Alignment)	210	300	65	575	9.6
LLama3.1-8b-Instruct (Self-Canonicalization)	200	280	52	532	8.9

Table 3.3: Summary of models with time breakdown for OIE, SD, and SC, total time in minutes, and total time in hours on Judgment titled Indian Oil Corporation v NEPC India Limited and Others - Supreme Court of India.” spread across 366 sentences.

LLM *Mistral-7B-Instruct-v0.2*

We started with **Mistral-7B-Instruct-v0.2**. As shown in Table 3.3, it took a total of 26.7 hrs. to generate the triplets for 366 lines of judgment. Triplets generated corresponding to above example sentence were:

```
[[‘NEPC India Ltd’, ‘instance of’, ‘first respondent’],
[‘Skyline NEPC Limited’, ‘instance of’, ‘sister company’],
[‘Indian Oil Corporation’, ‘product or material produced’, ‘aviation
turbine fuel’],
[‘Indian Oil Corporation’, ‘product or material produced’, ‘aviation
lubricants’]]
```

Overall, model was able to extract majority of the information from the sentences semantically correct. But it was seen that the model was hallucinating and bringing outer context into some of the the generated triplets, which was in no sense related to the judgement.

LLM *LLama2-7b-chat-hf*

We then changed our model from Mistral to LLama2-7b-chat-hf. As shown in Table 3.3, it took a total of 22.5 hrs. which was better than Mistral’s time. Triplets generated corresponding to above example sentence were:

```
[[‘Indian Oil Corporation’, ‘instance of’, ‘IOC’],
[‘Skyline NEPC Limited’, ‘instance of’, ‘sister company’]]
```

We observed that though the generation time of this model was better than Mistral, but it was not performing better in the sense of quality, as the triplets were much sparser and contained less rich information than mistral was generating. The second issue was that it, like Mistral, was also hallucinating.

LLM *LLama3.1-8b-Instruct* (Target Alignment)

We then moved forward and used LLama3.1-8b-Instruct model for triplets generation. As shown in Table 3.3, this time it took only 9.6 hours. Triplets generated corresponding to above sentence were:

□

The time reduced significantly, but the results were even poorer than the previous two models (as can be seen in case of above example, no triplets were generated).

We tracked back in our pipeline to check the fault and found out that we were using the schema of REBEL dataset in the Canonicalization step. This form of canonicalization was named **TARGET ALIGNMENT** in [16], where we provide a schema additionally with the dataset, like the traditional triplet generation techniques we discussed in chapter-2. That's why the models performed poorly, because the schema was not correlated with the judgment dataset and its terminologies. The [16] did provide an alternative approach for this, named **SELF CANONICALIZATION** in which it generates schema on its own.

LLM *LLama3.1-8b-Instruct* (Self-Canonicalization)

So we regenerated the triplets using same model but using self canonicalization approach this time. As shown in Table 3.3, it took a total of 8.9 hrs. and the triplets generated corresponding to above sentence were:

```
[[['Indian Oil Corporation', 'entered into contract with', 'NEPC India  
Ltd'],  
 ['Indian Oil Corporation', 'entered into contract with', 'Skyline NEPC  
Limited'],  
 ['Skyline NEPC Limited', 'sister company of', 'Indian Oil Corporation']]
```

This time we had the lowest triplet generation time as well as the best quality of triplets among all the models. So, we decided to use this setting in our further triplet generation task on the legal documents.

Sentence level comparison, Mistral(7b) vs LLama3.1(8b)

If we ignore the triplet generation time and only focus on the quality of triplets generated by the models, we can say from the above experiments that *Mistral-7B-Instruct-v0.2* (*Target-Alignment*) and *LLama3.1-8b-Instruct* (*Self-Canonicalization*) extracted the best triplets among all. To compare both the models sentence wise we created a script and generated the text file containing following couples of triplet set.

1. Model 1:

[[['Indian Oil Corporation', 'relative', 'NEPC India Limited'], 'NEPC India Limited', 'relative', 'Others']]

Model 2:

[[['Indian Oil Corporation', 'v', 'NEPC India Limited'], ['Indian Oil Corporation', 'v', 'Others']]

2. Model 1:

[[['Supreme Court of India', 'applies to jurisdiction', 'highest court'], ['Supreme Court of India', 'country', 'India']]

Model 2:

[[['Supreme Court of India', 'country', 'India'], ['Supreme Court of India', 'type of organization', 'court'], ['Supreme Court of India', 'location', 'India']]

3. Model 1:

[[['John Doe', 'place of birth', 'New York City'], ['New York City', 'country', 'United States'], ['John Doe', 'occupation', 'Software Engineer'], ['Software Engineer', 'subclass of', 'Engineer'], ['Engineer', 'field of work', 'Technology']]

Model 2:

[]

4. Model 1:

[]

Model 2:

[[['20 July 2006', 'date', 'event']]

5. Model 1:

[[['Cr.A', 'relative', 'unknown']]

Model 2:

[]

6. Model 1:

[[['No 834 of 2002', 'subclass of', 'Legislation'], ['Year 2002', 'point in time', '2002']]

Model 2:

[[['No 834 of 2002', 'type of legal document', 'Act'], ['Act', 'year of

enactment', '2002']]

7. Model 1:

[['R V Raveendran', 'instance of', 'Judge']]

Model 2:

[['R V Raveendran', 'delivered judgment', 'The Judgment']]

In the above 7 examples, **Model 1** corresponds to Mistral(7b) and **Model 2** corresponds to LLama3.1(8b). The text in **red colour** is the hallucinated text generated by Mistral(7b).

3.3.3 Triplets Generation of 10 Judgment Summaries

LLM LLama3.1-8b-Instruct (Self-Canonicalization)

Once the model was selected, we proceeded to our next task, which was to generate triplets corresponding to 10 judgment summaries. The Information about summary filename, no. of sentences in the corresponding summary, time taken at each step (OIE, SD, SC) on SYSTEM 1 and SYSTEM 2, and total time is provided in Table-3.4.

Filename (.txt)	No. of sentences	SYS 1's time (mins)			SYS 2's time (mins)			Total time (mins)
		OIE	SD	SC	OIE	SD	SC	
64813_extracted	21	11	15	4	16	25	6	30, 47
64813_gold	23	13	24	4	19	38	7	41, 64
745161_extracted	21	-	-	-	21	35	7	63
745161_gold	38	25	37	5	-	-	-	67
1114158_extracted	25	-	-	-	20	37	7	64
1114158_gold	38	19	29	7	-	-	-	55
1316559_extracted	55	36	50	9	-	-	-	95
1316559_gold	53	-	-	-	51*	77*	14*	140*
1363234_extracted	30	18	25	6	-	-	-	50
1363234_gold	26	17	30	5	-	-	-	52

Table 3.4: Time taken by SYSTEM 1 and SYSTEM 2 for each step (OIE, SD, and SC) corresponding to each Document.(* denotes running of another script parallely on the system, impacting increase in the generation time)

Chapter 4

Conclusion and Future Work

In this report, we have compared multiple LLMs for the Triplet Generation task by using the EDC technique introduced by [16]. We found *LLama3.1-8b-Instruct* to be the capable model for extracting the relevant triplets corresponding to the Legal Domain. Self-Canonicalization approach was the key factor behind the good quality of triplets generated.

Till now we have only utilised one pipeline and that leaves us with its drawbacks too. The major drawback of using the EDC technique is the Inference time it takes to Extract, Define and then Canonicalize the Triplets. That is why this approach is not much scalable to extract triplets of large number of judgments and other legal documents. In the future we aim to explore more pipelines and compare both the inference time and the quality of triplets generated.

In the future work, we aim to develop a novel approach for Information extraction and knowledge graph construction especially for legal domain that addresses the limitations of EDC. Our main objective would be to harness the power of pretrained LLMs and prompt engineering to produce better quality triplets.

References

- [1] L. Han, T. Finin, C. Parr, J. Sachs, and A. Joshi, “Rdf123: From spreadsheets to rdf,” in *The Semantic Web - ISWC 2008* (A. Sheth, S. Staab, M. Dean, M. Paolucci, D. Maynard, T. Finin, and K. Thirunarayan, eds.), (Berlin, Heidelberg), pp. 451–466, Springer Berlin Heidelberg, 2008.
- [2] W. Ammar, D. Groeneveld, C. Bhagavatula, I. Beltagy, M. Crawford, D. Downey, J. Dunkelberger, A. Elgohary, S. Feldman, V. Ha, R. Kinney, S. Kohlmeier, K. Lo, T. Murray, H. Ooi, M. Peters, J. Power, S. Skjonsberg, L. Wang, C. Wilhelm, Z. Yuan, M. Van Zuylen, and O. Etzioni, “Construction of the literature graph in semantic scholar,” in *Industry Papers, NAACL HLT 2018 - 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference*, pp. 84–91, Association for Computational Linguistics (ACL), Jan. 2018. 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL HLT 2018 ; Conference date: 01-06-2018 Through 06-06-2018.
- [3] M. Faerber, *The Microsoft Academic Knowledge Graph: A Linked Data Source with 8 Billion Triples of Scholarly Data*, pp. 113–129. 10 2019.
- [4] E. Riloff and R. Jones, “Learning dictionaries for information extraction by multi-level bootstrapping,” pp. 474–479, 01 1999.
- [5] E. Agichtein and L. Gravano, “Snowball: extracting relations from large plain-text collections,” in *Proceedings of the Fifth ACM Conference on Digital Libraries*, DL ’00, (New York, NY, USA), p. 85–94, Association for Computing Machinery, 2000.
- [6] L. Page, S. Brin, R. Motwani, and T. Winograd, “The pagerank citation ranking : Bringing order to the web,” in *The Web Conference*, 1999.
- [7] A. Yates, M. Banko, M. Broadhead, M. Cafarella, O. Etzioni, and S. Soderland, “TextRunner: Open information extraction on the web,” in *Proceedings of Human Language Technologies: The Annual Conference of the North American Chap-*

- ter of the Association for Computational Linguistics (NAACL-HLT) (B. Carpenter, A. Stent, and J. D. Williams, eds.), (Rochester, New York, USA), pp. 25–26, Association for Computational Linguistics, Apr. 2007.
- [8] F. Wu and D. S. Weld, “Open information extraction using Wikipedia,” in *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics* (J. Hajič, S. Carberry, S. Clark, and J. Nivre, eds.), (Uppsala, Sweden), pp. 118–127, Association for Computational Linguistics, July 2010.
 - [9] Mausam, M. Schmitz, S. Soderland, R. Bart, and O. Etzioni, “Open language learning for information extraction,” in *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning* (J. Tsujii, J. Henderson, and M. Paşca, eds.), (Jeju Island, Korea), pp. 523–534, Association for Computational Linguistics, July 2012.
 - [10] K. Gashteovski, R. Gemulla, and L. del Corro, “MinIE: Minimizing facts in open information extraction,” in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing* (M. Palmer, R. Hwa, and S. Riedel, eds.), (Copenhagen, Denmark), pp. 2630–2640, Association for Computational Linguistics, Sept. 2017.
 - [11] C. Niklaus, M. Cetto, A. Freitas, and S. Handschuh, “A survey on open information extraction,” in *Proceedings of the 27th International Conference on Computational Linguistics* (E. M. Bender, L. Derczynski, and P. Isabelle, eds.), (Santa Fe, New Mexico, USA), pp. 3866–3878, Association for Computational Linguistics, Aug. 2018.
 - [12] D. Xu, W. Chen, W. Peng, C. Zhang, T. Xu, X. Zhao, X. Wu, Y. Zheng, Y. Wang, and E. Chen, “Large language models for generative information extraction: A survey,” 2024.
 - [13] X. Wei, X. Cui, N. Cheng, X. Wang, X. Zhang, S. Huang, P. Xie, J. Xu, Y. Chen, M. Zhang, Y. Jiang, and W. Han, “Chatie: Zero-shot information extraction via chatting with chatgpt,” 2024.
 - [14] Z. Bi, J. Chen, Y. Jiang, F. Xiong, W. Guo, H. Chen, and N. Zhang, “Codekgc: Code language model for generative knowledge graph construction,” 2024.
 - [15] S. Wadhwa, S. Amir, and B. Wallace, “Revisiting relation extraction in the era of large language models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (A. Rogers, J. Boyd-Graber, and N. Okazaki, eds.), (Toronto, Canada), pp. 15566–15589, Association for Computational Linguistics, July 2023.

- [16] B. Zhang and H. Soh, “Extract, define, canonicalize: An llm-based framework for knowledge graph construction,” *arXiv preprint arXiv:2404.03868*, 2024.
- [17] P.-L. Huguet Cabot and R. Navigli, “REBEL: Relation extraction by end-to-end language generation,” in *Findings of the Association for Computational Linguistics: EMNLP 2021* (M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, eds.), (Punta Cana, Dominican Republic), pp. 2370–2381, Association for Computational Linguistics, Nov. 2021.

Appendix A

Experiments Output Files

The datasets and the generated triplet files of the project are available on GitHub ¹.

The repository has been organised as follows:

- **REBEL/**
 - **19 Sentences/** contains the dataset and triplets generated on 19 Sentences
 - **100 Sentences/** contains the dataset and triplets generated on 100 Sentences
 - **1000 Sentences/** contains the dataset and triplets generated on 1000 Sentences
- **The Judgement & Experiments/**
 - **The Judgment Dataset/** contains the Judgment dataset, both raw and tokenized.
 - **Models & Outputs/** contains the Triplets generated using 4 different LLMs and approaches.
- **10 Judgment Summaries/**
 - **Summaries/** contains preprocessed summaries of 10 different Judgments.
 - **Output/** contains the triplets generated corresponding to the summaries using LLama3.1-8b (self-canonicalization).

All the above folder contains one extra file "Combined File" consisting of original sentences attached with generated triplets which is helpful for Analysis.

¹<https://github.com/Harsh-Yadav-02/LegalKG>